Product Requirements Document (PRD): meeco

1. Overview

meeco is a mobile-first voice recording, transcription, and action-extraction web app (`frontend/` React + Vite, `backend/` FastAPI). A user taps record, speaks, and gets a clean transcript, an AI-generated title, an on-demand summary, checkable action items, and an A4 PDF export.

Phased Delivery Strategy

This PRD is structured for a **2-engineer beginner AI team** shipping in two clear stages:

- **Phase 1 (Primary Target — Zero AssemblyAI):** Free live transcription in the browser via the **Web Speech API**, paired with a stateless **FastAPI + Groq LLM** backend for real AI titles, summaries, action items, and cross-session history.
- **Phase 2 (Optional Production Upgrade):** Add AssemblyAI streaming + S3 backup chunking & timeline gap repair to solve browser gaps (Firefox), silence/network drops, and 2h30m meeting reliability.

2. Team Ownership (Phase 1)

- **Engineer A (Client Speech & API Integration):** Browser Web Speech transcription (`transcriber.js`), local fallback heuristics (`mockActions.js`), and bridging Web Speech transcripts + recording duration to the backend API so Groq handles post-processing while keeping local audio playback.
- **Engineer B (Backend AI & Session Storage):** Groq LLM integration (`llm.py` — prompts, JSON mode, retries, model fallback) and stateless session lifecycle (`sessions.py` — create, finalize, title, summary, action items, history CRUD).

(See INTENT.md for the architectural intent, invariants, and exact repo restriction split.)

3. Operating Modes

Mode	Trigger	Transcription	Summary / Action Items	Persistence
On-device (Standalone)	No `VITE_API_URL` and no `?api=`	Browser Web Speech API (`SpeechRecognition`) with demo fallback	Local extractive heuristics (`mockActions.js`)	`localStorage`
Phase 1 Hybrid (Web Speech + Groq API)	`VITE_API_URL` or `?api=<url>` (with `TRANSCRIBER=fake`)	Browser Web Speech API submitted on Stop as `owned: [[0, duration_ms]]`	Groq LLM (`llama-3.3-70b-versatile` + fallback)	Object storage (`local` or `s3`)
Phase 2 Full Cloud	`VITE_API_URL` + `TRANSCRIBER=assemblyai`	AssemblyAI live WebSocket + S3 chunk gap repair	Groq LLM	Object storage (`local` or `s3`)
Demo	`VITE_DEMO=1`	Simulated live captions (`DEMO_TRANSCRIPT`)	Local extractive heuristics	`localStorage`

4. Phase 1 Functional Requirements (No AssemblyAI)

FR-1: Browser Capture & Live Web Speech Transcription (transcriber.js)

- **FR-1.1 Mic & Playback Capture:** Pressing Record opens mic capture (`getUserMedia`) for the 60 fps 9-bar `Waveform` meter (`meter.js`) and `MediaRecorder` (`WebM/Opus` or `MP4/AAC`) so the user can play back the last recording locally (`usePlayer`).
- **FR-1.2 Live Captions:** Use `window.SpeechRecognition || window.webkitSpeechRecognition` (`continuous = true`, `interimResults = true`, `lang = navigator.language || 'en-US'`). Emit `(finalText + interim).trim()` live as the user speaks.
- **FR-1.3 Chrome Auto-Restart:** Chrome stops "continuous" recognition on pauses. Restart automatically on `onend` while recording unless recognition fails rapidly 4 times in a row (`< 400 ms` per attempt -> `'unstable'`).
- **FR-1.4 Graceful Fallback:**
  - Stop cleanly on Stop with a `2500 ms` flush timeout and a `1700 ms` minimum processing-ring duration so the UI never flashes.
  - On unsupported browsers (e.g., Firefox) or fatal recognition errors (`not-allowed`, `service-not-allowed`, `audio-capture`, `network`, `language-not-supported`) with empty text, return `DEMO_TRANSCRIPT` with a clear `notice` banner.

FR-2: Client-to-Backend Transcript Handoff (cloudEngine.js / useRecorder.js)

- **FR-2.1 Zero Audio Streaming Needed:** In Phase 1, do not open AssemblyAI WebSockets or upload audio chunks to S3. Keep `MediaRecorder` running locally for playback.
- **FR-2.2 Finalize Payload:** On Stop, call `POST /v1/sessions` (if not already created) and `POST /v1/sessions/{sid}/finalize` with:

```
{
  "duration_ms": 12000,
  "owned": [[0, 12000]],
  "turns": [{ "start_ms": 0, "end_ms": 12000, "text": "..." }]
}
```

Because `owned` covers `[0, duration_ms]`, the backend finds `0` gaps and skips batch ASR completely.

- **FR-2.3 Offline Fallback:** If no API URL is configured, run completely on-device using `mockActions.js` and `localStorage`.

FR-3: Groq LLM Intelligence (llm.py)

- **FR-3.1 Language-Matched Title (`head_tokens` + `clean_title`):** Generate a max 6-word title in the same language as the transcript using `groq_model_title` (`llama-3.1-8b-instant`). Strip surrounding quotes, markdown symbols, and trailing periods; fall back to the first 6 words if empty or if the LLM fails.
- **FR-3.2 Structured Summary (`POST /v1/sessions/{sid}/summary`):** Return `{ overview: string, points: string[] }` (1–2 sentence overview, max 6 key points, same language, strictly grounded in the transcript). Cache on the stored session unless `refresh=true`.
- **FR-3.3 Action Item Extraction (`POST /v1/sessions/{sid}/action-items`):** Return `{ items: [{ id, text, done }] }` (max 10 imperative tasks actually committed to or requested, empty list if none). Persist checkbox state updates via `PUT /v1/sessions/{sid}/action-items`.
- **FR-3.4 Reliability & Fallbacks:**
  - Use OpenAI-compatible `response_format: {"type": "json_object"}` and tolerate markdown-fenced JSON via `_json_from()`.
  - Retry `429` and `5xx` errors (up to 3 attempts with exponential backoff).
  - If Groq refuses the primary model (`400`/`403`/`404` mentioning `"model"`), fall back once to

`groq_model_fallback` (`openai/gpt-oss-120b`, setting `reasoning_effort: "low"` and extra token headroom).

- **FR-3.5 No-Key Dev Double (`FakeLLM`):** Provide deterministic heuristic implementations for title, summary, and action items when `LLM_PROVIDER=fake`.

FR-4: Stateless Session & History Service (sessions.py)

- **FR-4.1 Device Scoping:** Scope every storage key by `X-Device-Id` (`^[A-Za-z0-9_-]{16,64}$`). Store zero session state in Python memory.
- **FR-4.2 Artifacts Stored (`LocalStorage` in dev, `S3Storage` in prod):**
  - `sessions/{device}/{sid}/meta.json` — session creation metadata
  - `sessions/{device}/{sid}/title.json` — early/cached title
  - `transcripts/{device}/{sid}.json` — final result (`id`, `title`, `created_at`, `duration_ms`, `transcript`, `turns`, `summary`, `action_items`, `warnings`, `recovered_ms`)
  - `transcripts/{device}/{sid}.meta.json` — compact row for fast history listing
- **FR-4.3 History CRUD:**
  - `GET /v1/sessions` returns compact `.meta.json` rows sorted newest-first.
  - `GET /v1/sessions/{sid}` returns the full result JSON (`404` if missing or wrong device).
  - `DELETE /v1/sessions/{sid}` deletes result, row, session folder, and any audio prefix.
- **FR-4.4 Idempotent Finalize:** Repeating `POST /v1/sessions/{sid}/finalize` returns the already-saved result without re-running the LLM.

FR-5: PDF Export & Single-File Distribution (Provided Shell)

- Export title, timestamp/duration/word metadata, summary, checkboxed action items, and paragraphed transcript to an A4 PDF (`pdf.js` via `jsPDF`) with Web Share API or download fallback.

---

5. Phase 2 Requirements (Upgrade Path: AssemblyAI + S3 Safety Net)

Once Phase 1 is working end-to-end, upgrade the transcription layer to remove Web Speech limits (no Firefox support, no recovery on network drop, variable mobile stability):

- **Live Streaming (`liveStream.js` + `pcm-worklet.js`):** Stream 100 ms 16 kHz PCM frames straight from browser to AssemblyAI v3 WebSocket (`universal-3-6-pro`, `voice_focus=near-field`) using short-lived backend-minted tokens (`mint_stream_token`), replaying up to `5 s` on reconnect and offsetting timestamps onto `[0, duration_ms]`.
- **S3 Backup Chunks (`chunker.js` + `oggOpus.js`):** Compress 30 s audio chunks to Ogg Opus (`24 kbps`, ~3 KB/s, WAV fallback) and `PUT` directly to S3 presigned URLs.
- **Timeline Ledger & Gap Repair (`ledger.py`, `audio.py`, `sessions.py`):** Send 15 s checkpoints (triggering early title generation at ~300 tokens), compare `owned` intervals against `duration_ms` (`find_gaps`), skip batch ASR on clean recordings, and batch-transcribe only missing spans via `AssemblyAITranscriber`.

---

6. API Surface (/v1)

- `GET /health`
- `POST /v1/sessions` — create session
- `POST /v1/sessions/{sid}/finalize` — save transcript + generate title (Phase 1 & 2)
- `GET /v1/sessions`, `GET /v1/sessions/{sid}`, `DELETE /v1/sessions/{sid}` — history
- `POST /v1/sessions/{sid}/summary`, `POST /v1/sessions/{sid}/action-items`, `PUT`

`/v1/sessions/{sid}/action-items` — AI actions

- *(Phase 2 endpoints already mounted in `routes/sessions.py`):* `POST /stream-token`, `POST /chunks`, `PUT /checkpoint`

---

7. Acceptance Criteria

Phase 1 Done Criteria

1. **Live Browser Recording:** Recording in Chrome/Edge at `http://localhost:5173/?api=http://localhost:8000` shows live Web Speech captions, preserves local playback, and saves the transcript on the backend.
2. **Groq AI Outputs:** With `LLM_PROVIDER=groq` and `TRANSCRIBER=fake`, Stop produces a clean <=6-word title, Summary tab generates valid JSON `{ overview, points }`, Action Items tab extracts tasks and persists checkbox toggles across reload, and zero AssemblyAI keys or calls are required.
3. **Backend Tests:**
   - `pytest tests/test_providers.py -k "groq or title"` passes (mocked retries, fenced JSON, model fallback).
   - `pytest tests/test_api.py -k "device or unknown or happy_path or history"` passes.

Phase 2 Done Criteria (If Enabled Later)

- Full `cd frontend && npm test`, full `cd backend && pytest`, and `python e2e/run.py clean|drop|gap` pass.